Parallel Sorting and Complexity

Odd-even transposition sort, parallel quicksort and parallel merge sort

Slide summary: algorithms, implementations and work-span analysis

Outline

1 **Complexity toolkit**
work-span model and sequential baselines — slides 3–4

2 **Odd-even transposition sort**
idea, trace, implementations, complexity — slides 5–8

3 **Parallel quicksort**
tasks, parallel partition, complexity — slides 9–11

4 **Parallel merge sort**
tasks, parallel merge, complexity — slides 12–14

5 **Comparison and speedup**
side-by-side table and speedup model — slides 15–16

6 **Cheat sheet**
the formulas to remember — slide 17

Notation used in this deck

n = number of keys to sort · **p** = number of processors (or threads) · **W** = work (total operations) · **D** = span (critical path) · T_p = running time on p processors · log = $\log_2$. **Θ** is a tight bound, **O** an upper bound and **Ω** a lower bound. Bounds marked with * are **expected** values (random pivot).

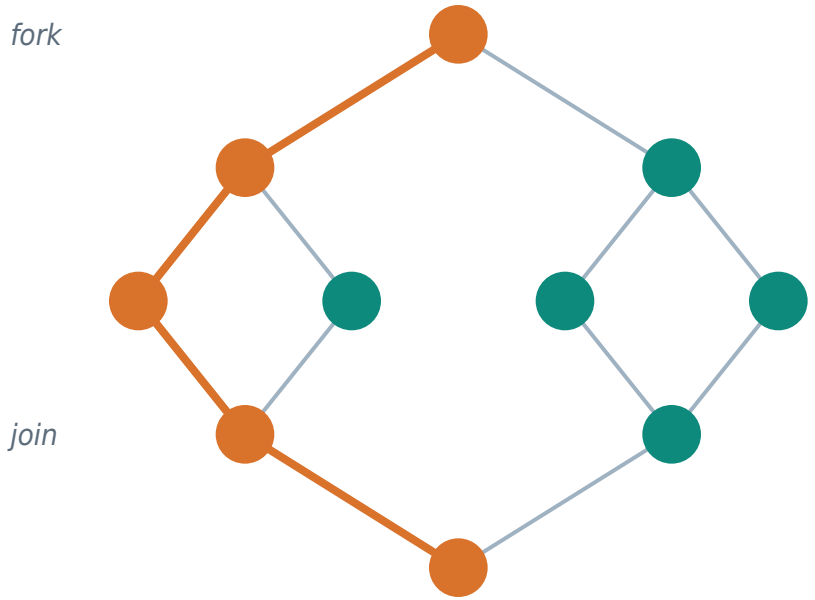
Complexity toolkit: the work-span model

- **Work** $W = T_1$: total number of operations (the time on a single processor)
- **Span** $D = T_\infty$: the longest chain of dependent operations (the critical path); the time with unlimited processors
- **Parallelism** = W / D: the largest number of processors that can be kept busy
- With p processors (greedy scheduler): $\max(W/p, D) \le T_p \le \mathbf{W/p + D}$

Speedup	Efficiency	Cost
$\mathbf{S = T_1 / T_p}$	$\mathbf{E = S / p}$	$\mathbf{p \cdot T_p}$

Cost-optimal if $p \cdot T_p = \Theta(T_{seq})$ of the best sequential algorithm. For comparison sorting that is $\Theta(n \log n)$.

a fork-join computation as a DAG



W = 10 nodes (work) · **D = 5** nodes on the longest path (span) · parallelism W/D = 2

Sequential baselines

Algorithm	Best	Average	Worst	Extra memory	Parallelism comes from
Odd-even transposition (sequential)	$\Theta(n)$ with early exit	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	independent adjacent compare-exchanges
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$ stack	two recursive calls; the partition step
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	two recursive calls; the merge step

Lower bound: any comparison-based sort needs $\Omega(n \log n)$ comparisons, so a parallel sort is **cost-optimal** only when $p \cdot T_p = \Theta(n \log n)$.

Plan: for each algorithm we (1) find the independent work, (2) implement it with OpenMP tasks or loops, and (3) compute **W, D, T_p and the cost**.

Odd-even transposition sort: the idea

- A variant of **bubble sort** that compares **disjoint** pairs, so the comparisons can run in parallel
- It alternates two phases. **Even phase:** compare-exchange (a[0],a[1]), (a[2],a[3]), ... **Odd phase:** compare-exchange (a[1],a[2]), (a[3],a[4]), ...
- **Compare-exchange:** swap the two keys only if they are out of order
- After **n phases** the array is guaranteed to be sorted
- Sorting network style: fixed communication pattern, only neighbors talk to each other

> **Key point:** the pairs inside one phase never overlap, so no locks are needed. Only a **barrier between phases** is required.

Pseudocode

```
for phase in 0 .. n-1:
    start = phase mod 2     // 0: even, 1: odd
    for i = start; i < n-1; i += 2:  // parallel
        compare_exchange(a[i], a[i+1])
    barrier                 // end of the phase
```

Even phase: pairs (0,1) (2,3) (4,5) (6,7)



Odd phase: pairs (1,2) (3,4) (5,6), ends are idle



Odd-even transposition sort: trace

start	7	3	8	1	6	2	5	4
phase 0 (even)	3	7	1	8	2	6	4	5
phase 1 (odd)	3	1	7	2	8	4	6	5
phase 2 (even)	1	3	2	7	4	8	5	6
phase 3 (odd)	1	2	3	4	7	5	8	6
phase 4 (even)	1	2	3	4	5	7	6	8
phase 5 (odd)	1	2	3	4	5	6	7	8
phase 6 (even)	1	2	3	4	5	6	7	8
phase 7 (odd)	1	2	3	4	5	6	7	8

Legend

- pair compared and swapped
- pair compared, already in order
- idle in this phase

Result: the array is sorted after **6 phases** here, but the guarantee is **n = 8** phases (worst case).

Why up to n phases? a key moves at most **one position per phase**. In a reversed array the smallest key must travel n - 1 positions.

Odd-even transposition sort: implementations

Shared memory (OpenMP)

```
#pragma omp parallel      // one team, all phases
for (int phase = 0; phase < n; phase++) {
    #pragma omp for
    for (int i = phase % 2; i < n - 1; i += 2)
        if (a[i] > a[i + 1])
            swap(&a[i], &a[i + 1]);
    // implicit barrier at the end of omp for
}
```

The **implicit barrier** at the end of `omp for` separates the phases, and the team is created **once** instead of n times.

Putting `parallel for` inside the phase loop also works, but pays one fork-join (overhead!) per phase.

Distributed memory (p processes)

1. Each process holds n/p keys and sorts them **locally**.

2. For p phases, neighbors exchange blocks and **merge-split**: the lower rank keeps the smaller n/p keys, the higher rank the larger n/p.

3. After p phases the blocks are globally sorted.

Even phase: P0 ↔ P1 and P2 ↔ P3



Odd phase: P1 ↔ P2 (P0 and P3 idle)



Odd-even transposition sort: complexity

Execution model	Time T_p	Work / cost
p = n/2 (one processor per pair)	n phases × O(1) = **Θ(n)**	W = n × n/2 = Θ(n^2); cost p·T_p = Θ(n^2): **not** cost-optimal
shared memory, p < n threads	n × (n/(2p) + barrier) = **Θ(n^2/p)** + n barriers	cost = Θ(n^2) while p ≤ n
distributed, p blocks of n/p keys	**Θ((n/p) log(n/p) + n)**: local sort + p phases of Θ(n/p) compute and Θ(n/p) communication	cost = Θ(n log(n/p) + p·n): cost-optimal **only if p = O(log n)**

Break-even: $n^2/(2p) + n < n \log n \Rightarrow p > n / (2(\log n - 1))$. For $n = 2^{20}$ that is about **27,600 processors** before it beats sequential merge sort.

Good news: O(1) extra memory, **stable** (it only swaps strictly out-of-order pairs), and communication is neighbor-only, which suits linear arrays and meshes.

Takeaway: odd-even transposition is simple and has a perfect fixed pattern, but it does Θ(n^2) work. The parallel time Θ(n) is short only because it uses **many processors**.

Parallel quicksort: tasks

- **Sequential recap:** pick a pivot, partition into smaller / larger keys, sort both sides recursively
- **Parallelism 1:** the two recursive calls are **independent** (disjoint sub-arrays), so each becomes an `omp task`
- **Parallelism 2:** the partition itself can be parallelized (next slide)
- A `CUTOFF` switches to sequential code for small ranges: it keeps the **granularity** high

> **Limit:** the partition at the root is still a **sequential Θ(n) step**. While it runs, only one thread works, and the next level has only two tasks.

Task-parallel quicksort

```
void pqsort(int *a, int lo, int hi) {
    if (hi - lo < CUTOFF) {
        seq_qsort(a, lo, hi);   // go sequential
        return;
    }
    int p = partition(a, lo, hi); // O(n), serial
    #pragma omp task
    pqsort(a, lo, p - 1);
    #pragma omp task
    pqsort(a, p + 1, hi);
}
```

```
#pragma omp parallel
#pragma omp single      // one thread starts it
pqsort(a, 0, n - 1);
```

No `taskwait` needed: the barrier at the end of the parallel region waits for all tasks.

Parallel quicksort: partition with prefix sums

index i	0	1	2	3	4	5	6	7
a[i]	6	2	9	3	8	1	7	4
flag: a[i] < 5	0	1	0	1	0	1	0	1
prefix sum of flags	0	0	1	1	2	2	3	3
destination	4	0	5	1	6	2	7	3
result	2	3	1	4	6	9	8	7

Pivot: value 5 (e.g. the median of a small sample). Keys < 5 go first, the rest after them, keeping their order (**stable**).

Destination: `scan[i]` if flag = 1, else `L + i - scan[i]`, with L = 4 (number of flags).

1. Flag

every key is compared with the pivot independently

span Θ(1), work Θ(n)

2. Scan

parallel prefix sum gives each key its rank among the flagged keys

span Θ(log n), work Θ(n)

3. Scatter

every key is written to its destination independently

span Θ(1), work Θ(n)

Whole partition: work Θ(n) and span **Θ(log n)** instead of a sequential Θ(n). The price is Θ(n) extra memory for the flags, the scan and the output array.

Parallel quicksort: complexity

Variant	Work W	Span D	Parallelism W/D	Time T_p
Sequential	Θ(n log n)*	Θ(n log n)*	1	Θ(n log n)*
Tasks + sequential partition	Θ(n log n)*	Θ(n)*	Θ(log n)	O(n log n / p + n)
Tasks + parallel partition	Θ(n log n)*	$\Theta(\log^2 n)$*	Θ(n / log n)	$O(n \log n / p + \log^2 n)$

** expected with a random pivot (or median-of-3); with bad pivots the work degrades to Θ(n²).*

Where the span comes from

- Sequential partition: D(n) = D(larger side) + Θ(n). The sizes shrink geometrically (expected 3/4), so D = Θ(n + 3n/4 + ...) = **Θ(n)**
- Parallel partition: every level costs Θ(log n) span, and the expected recursion depth is Θ(log n), so D = Θ(log n) × Θ(log n) = **$\Theta(\log^2 n)$**
- Work: W(n) = W(k) + W(n-k-1) + Θ(n) = Θ(n log n) expected

Distributed memory: split the processes into two groups at every pivot (log p rounds): local partition, exchange, recurse. With balanced pivots the compute is about (n/p) log n, plus Θ(n/p) communication per round.

Parallel merge sort: tasks

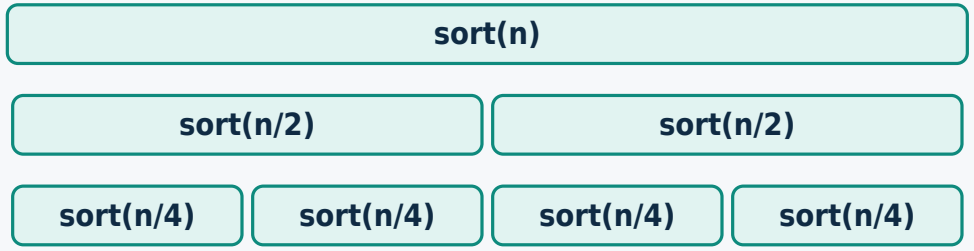
Task-parallel merge sort

```
void pmsort(int *a, int *tmp, int lo, int hi) {
    if (hi - lo < CUTOFF) {
        seq_sort(a, lo, hi);        // small: serial
        return;
    }
    int mid = lo + (hi - lo) / 2;
    #pragma omp task
    pmsort(a, tmp, lo, mid);
    #pragma omp task
    pmsort(a, tmp, mid, hi);
    #pragma omp taskwait  // children must finish
    merge(a, tmp, lo, mid, hi);   // serial O(n)
}
```

```
#pragma omp parallel
#pragma omp single
pmsort(a, tmp, 0, n);
```

- **Divide:** split the range in two halves
- **Conquer:** sort both halves in parallel, one task each (disjoint ranges: no race)
- **Combine:** merge the sorted halves, but only **after** `taskwait`
- The `tmp` buffer costs Θ(n) extra memory

> **IMPORTANT:** here `taskwait` is **required**. Without it the merge could read halves that are still being sorted.



tasks fork downward; merges join upward after taskwait

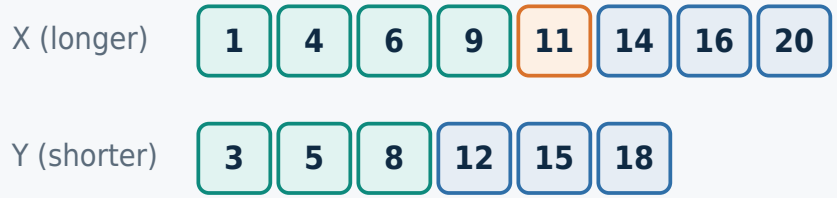
Parallel merge: split by binary search

- **1.** Let X be the **longer** array (|X| ≥ |Y|). Take its middle key x = X[m]
- **2.** Binary-search Y for the number j of keys smaller than x: O(log n)
- **3.** Key x goes to output index **m + j**: its final position
- **4.** Merge (X[0..m), Y[0..j)) and (X(m..], Y[j..]) **in parallel**: independent, with disjoint output ranges
- Each subproblem has at most **3/4** of the keys, because X holds at least half of them

Recursive parallel merge (serial merge below a cutoff)

```
pmerge(X, Y, out):
    if |X| < |Y|: swap(X, Y)      // X is longer
    if |X| == 0: return
    m = |X| / 2;  j = lower_bound(Y, X[m])
    out[m + j] = X[m]
    task: pmerge(X[0..m),  Y[0..j),  out)
    task: pmerge(X(m..],   Y[j..],  out + m + j + 1)
    taskwait
```

Example



x = X[4] = 11. Binary search in Y: 3 keys are smaller than 11, so j = 3 and x goes to index 4 + 3 = 7.

Output (14 keys)



task A
merge([1,4,6,9], [3,5,8])
→ indices 0..6

task B
merge([14,16,20], [12,15,18])
→ indices 8..13

> **Result:** parallel merge has work **Θ(n)** (same as the sequential merge) and span **Θ(log^2 n)**: M(n) = M(3n/4) + Θ(log n). Both sub-merges are just two more tasks.

Parallel merge sort: complexity

Variant	Work W	Span D	Parallelism W/D	Time T_p
Sequential	$\Theta(n \log n)$	$\Theta(n \log n)$	1	$\Theta(n \log n)$
Tasks + sequential merge	$\Theta(n \log n)$	$\Theta(n)$	$\Theta(\log n)$	$O(n \log n / p + n)$
Tasks + parallel merge	$\Theta(n \log n)$	$\Theta(\log^3 n)$	$\Theta(n / \log^2 n)$	$O(n \log n / p + \log^3 n)$

All bounds hold in the worst case (no random pivot involved). Extra memory: $\Theta(n)$.

Recurrences

- Work: $W(n) = 2\,W(n/2) + \Theta(n) = \mathbf{\Theta(n \log n)}$
- Sequential merge: $D(n) = D(n/2) + \Theta(n) = \mathbf{\Theta(n)}$
- Parallel merge: $D_{merge}(n) = D_{merge}(3n/4) + \Theta(\log n) = \Theta(\log^2 n)$
- Merge sort: $D(n) = D(n/2) + \Theta(\log^2 n) = \Sigma \log^2(n/2^i) = \mathbf{\Theta(\log^3 n)}$

Distributed memory: scatter, sort n/p keys locally, then log p merge rounds. The last round merges all n keys, so the rounds add up to $\Theta(n)$: $T_p = \Theta((n/p) \log(n/p) + n)$ unless the merges are parallelized too.

Side-by-side comparison

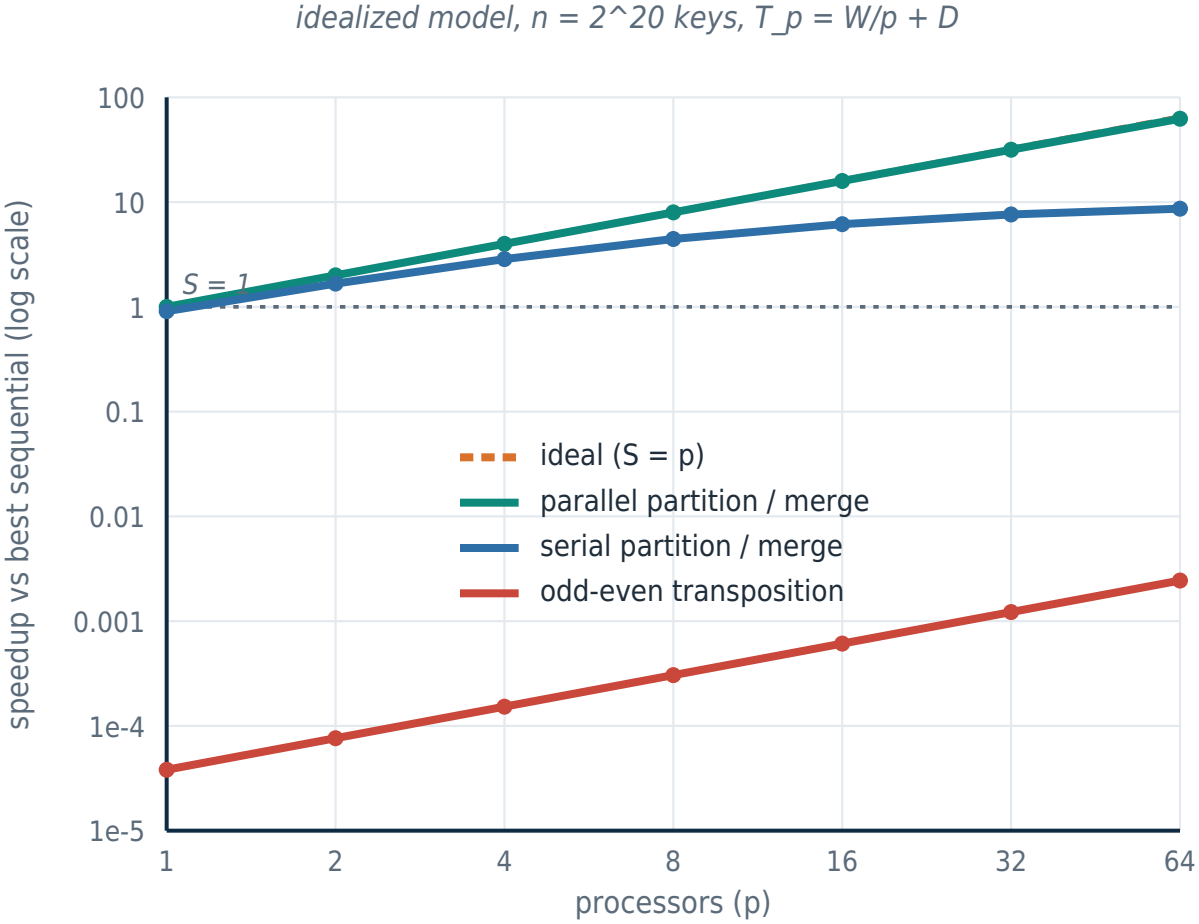
Algorithm	Work W	Span D	Parallelism	Extra memory	Stable	Best fit
Odd-even transposition (p = n/2)	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$	$O(1)$	yes	neighbor-only links, meshes
Quicksort + tasks (serial partition)	$\Theta(n \log n)$*	$\Theta(n)$*	$\Theta(\log n)$	$O(\log n)$	no	few cores, in place
Quicksort + parallel partition	$\Theta(n \log n)$*	$\Theta(\log^2 n)$*	$\Theta(n / \log n)$	$\Theta(n)$	can be	shared memory, many cores
Merge sort + tasks (serial merge)	$\Theta(n \log n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(n)$	yes	simple, few cores
Merge sort + parallel merge	$\Theta(n \log n)$	$\Theta(\log^3 n)$	$\Theta(n / \log^2 n)$	$\Theta(n)$	yes	worst-case guarantees

** expected with a random pivot. Parallelism = W / D. Sequential reference: Θ(n log n).*

Reading the table: odd-even transposition has the most parallelism but **wastes work** (n^2 instead of n log n). The task versions with a serial partition or merge are limited by a Θ(n) critical path. Only the **parallel partition** and **parallel merge** versions are work-efficient **and** have a short span.

Speedup model: why the baseline matters

Comparison



- **Model:** $T_p = W/p + D$, ignoring constants, memory bandwidth and caches. Real speedups are lower
- **Parallel partition / merge:** almost linear up to $p \approx W/D \approx 2{,}600$ processors
- **Serial partition / merge:** saturates at $W/D \approx (\log n)/2 = \mathbf{10\times}$ however many cores you add
- **Odd-even:** with $p \ll n$ it stays **below 1×**: slower than sequential merge sort until $p \approx 27{,}600$

Pitfall: always report speedup against the **best sequential algorithm**. Against sequential bubble sort, odd-even looks like a near-n× win, which is misleading.

Cheat sheet

Odd-even transposition

- n phases of disjoint compare-exchanges, barrier between phases
- p = n/2: $T = \Theta(n)$, $W = \Theta(n^2)$
- p blocks: $\Theta((n/p)\log(n/p) + n)$
- cost-optimal only for $p = O(\log n)$
- stable, O(1) extra memory

Parallel quicksort

- `omp task` for the two recursive calls, with a CUTOFF
- $W = \Theta(n \log n)$ expected
- serial partition: $D = \Theta(n)$
- parallel partition (scan): $D = \Theta(\log^2 n)$
- needs a good (random) pivot

Parallel merge sort

- `omp task` for the halves, `taskwait` before the merge
- $W = \Theta(n \log n)$, worst case
- serial merge: $D = \Theta(n)$
- parallel merge (binary search): $D = \Theta(\log^3 n)$
- stable, $\Theta(n)$ extra memory

Time on p processors
$T_p \approx W/p + D$

Speedup
$S = T_1 / T_p$

Efficiency
$E = S / p$

Cost
$p \cdot T_p$

Rule of thumb: start with the task versions and a CUTOFF (simple, good for a few cores). Move to **parallel partition** / **parallel merge** when you need to scale to many cores. Use odd-even transposition only when communication is restricted to neighbors.